

# Архиватор на базе алгоритма Хаффмана

Анпилогов Дарий Михайлович

Группа: 25.Б41-мм

Кафедра: системного программирования

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

## 1. Введение

Написание архиватора, основанного на базе алгоритма Хаффмана, — это учебная задача, направленная на получение опыта написания курсовой работы, изучения стандартных требований, процессов, а также способов достижения поставленных в рамках работы целей. Несмотря на наличие подобных решений, понимание проблем и требований при их разработке является важной частью обучения.

## 2. Постановка задачи

**Целью работы является** реализация консольного приложения, выполняющего в зависимости от параметров командной строки сжатие или разжатие файла по алгоритму Хаффмана.

**Для достижения цели требуется решить следующие задачи:**

1. Выполнить обзор существующих решений.
2. Выбрать инструменты для реализации.
3. Спроектировать решение.
4. Реализовать решение.
5. Выполнить экспериментальное исследование.

## 3. Обзор

Целью обзора является изучение алгоритма Хаффмана и существующих подходов к его реализации для выбора архитектурных решений разрабатываемого архиватора.

Алгоритм Хаффмана был предложен Дэвидом Хаффманом в 1952 году [1] и предназначен для сжатия данных без потерь. Алгоритм строит префиксный код переменной длины на основе частот появления символов во входных данных. Более часто встречающимся символам назначаются более короткие коды, а редким — более длинные. Для заданного набора частот алгоритм обеспечивает минимальную среднюю длину кодового слова среди всех префиксных кодов.

Алгоритм Хаффмана применяется как самостоятельный метод сжатия, а также используется в составе более сложных алгоритмов и форматов хранения данных. В частности, кодирование Хаффмана входит в состав алгоритма DEFLATE, применяемого в форматах ZIP, gzip и PNG [2], а также используется на этапе энтропийного кодирования в форматах JPEG и MP3.

При реализации архиватора необходимо решить две основные задачи: построение дерева Хаффмана и сохранение информации, необходимой для восстановления кодов при распаковке. Для построения дерева в большинстве реализаций используется очередь с приоритетами, позволяющая эффективно выбирать узлы с минимальными частотами. В данной работе используется контейнер `std::priority_queue`, обеспечивающий построение дерева за  $O(n \log n)$ , где  $(n)$  — количество различных символов входных данных. [3]

Для восстановления кодов при распаковке распространены два подхода: сохранение таблицы частот символов либо сохранение непосредственно кодов. В данной работе выбран второй вариант, поскольку он позволяет восстановить таблицу кодов без хранения полной таблицы частот и уменьшает объем служебной информации в архиве.

Таким образом, алгоритм Хаффмана является подходящей основой для реализации учебного архиватора благодаря сравнительно простой реализации, использованию эффективных структур данных и широкому распространению в современных форматах хранения данных, а также позволяет изучить работу с двоичными данными, файловым вводом-выводом, тестированием программного обеспечения и процессом разработки законченного программного решения.

## 4. Описание предлагаемого решения

Для реализации проекта были выбраны:

- Язык программирования: C++17. Причина — скорость и простота работы с потоками данных и битовыми операциями.
- Система сборки: CMake v3.14. Причина — простота сборки, кроссплатформенность.
- Тестирование: GoogleTest v1.14.0. Причина — стандарт индустрии, функциональность.

- CI: GitHub Actions. Причина — простота интеграции с платформой распространения проекта. Используется для запуска тестов и проверки программы с использованием clang-format, AddressSanitizer и UndefinedBehaviorSanitizer.

Реализация разделена на 2 части:

- Библиотека, реализующая функции архивации и разархивации.
- Консольное приложение, использующее библиотеку и предоставляющее пользователю интерфейс для использования функций.

Основными компонентами библиотеки являются:

- BitReader — чтение отдельных битов из входного потока с использованием внутреннего буфера;
- BitWriter — запись отдельных битов в выходной поток;
- Bits — контейнер для хранения битовых последовательностей;
- CodesTable — таблица кодов Хаффмана для всех возможных значений байта;
- CountTable — таблица частот встречаемости байтов (количества вхождений каждого байта);
- archive() — реализация алгоритма сжатия;
- unarchive() — реализация алгоритма разжатия.

Для построения кодов Хаффмана используется очередь с приоритетами `std::priority_queue`. Дерево в явном виде не строится. На каждой итерации алгоритма склеиваются два набора байтов с минимальными суммарными вхождениями в файл. К кодам соответствующих байтов добавляется по одному биту в зависимости от множества, к которому они принадлежат. Для разархивации явным образом строится дерево по таблице частот.

Архив содержит размер таблицы кодов, описание таблицы кодов, размер сжатых данных и сжатые данные. Не содержащиеся в исходном файле коды не хранятся в таблице кодов. Такой подход позволяет восстановить таблицу кодов без необходимости хранить полную таблицу частот.

Поддерживаются два режима работы архиватора. В режиме `twice` входной файл читается дважды: первый проход используется для подсчёта частот, второй — для кодирования данных. Данный режим позволяет обрабатывать файлы произвольного размера без полного размещения их содержимого в памяти. В режиме `ram` файл полностью загружается в оперативную память, что уменьшает количество операций ввода-вывода и позволяет работать с потоками, не поддерживающими повторное чтение.

Приложение поддерживает обработку ошибочных ситуаций: неверные аргументы командной строки, отсутствие входного файла, ошибки чтения и записи, а также повреждённые архивы. Для завершения работы используются различные коды возврата: 0 при успешном выполнении, 1 при ошибке параметров командной строки и -1 при прочих ошибках.

Для релизной версии дополнительно используются межпроцедурная оптимизация и оптимизации компилятора. Проект распространяется по лицензии Apache License 2.0.

Реализовано как модульное, так и интеграционное тестирование — всего 47 тестов.

## 5. Эксперименты

Для проведения экспериментального исследования были выбраны следующие инструменты:

- Язык программирования: Python 3.13. Причина — простота кода, совместимость с различными типами данных.
- Виртуальное окружение: `uv`. Причина — возможность явно указать используемые библиотеки, простой запуск.
- Формат сохранения данных: CSV. Причина — простота чтения и записи данных.
- Библиотека отрисовки графиков: `matplotlib`. Причина — стандарт для отрисовки графиков, простота использования, распространённость.

Для исследования производительности был разработан набор автоматизированных экспериментов. Измерения проводились на компьютере с процессором Intel Core i7-9750H, 16 ГиБ оперативной памяти под управлением операционной системы openSUSE Tumbleweed.

В качестве тестовых данных использовались:

- текстовые файлы;
- случайные данные;
- периодические последовательности с периодом 1, 2, 4, 8, 16 и 32 байта.

Размер файлов изменялся от 1 байта до 128 МиБ по степеням двойки. Для каждого измерения выполнялось 10 независимых запусков. Рассчитывались математическое ожидание и среднеквадратичное отклонение времени выполнения, а также скорости сжатия и разжатия.

Результаты показали, что степень сжатия существенно зависит от энтропии входных данных. Для периодических последовательностей коэффициент сжатия стремится к значению 0.125, что соответствует восьмикратному уменьшению размера файла. Для данных с высокой энтропией эффективность сжатия снижается, а в отдельных случаях размер архива может превышать размер исходного файла из-за накладных расходов на хранение таблицы кодов.

Максимальная скорость сжатия достигает порядка  $10^8$  байт в секунду. Скорость разжатия зависит от структуры данных сильнее: для случайных данных она составляет порядка  $10^7$  байт в секунду, а для периодических последовательностей может достигать  $1.75 \cdot 10^8$  байт в секунду.

Установлено, что рост энтропии данных приводит к почти линейному ухудшению коэффициента сжатия, а также к снижению скорости архивирования и разархивирования. Полученные зависимости представлены графиками, построенными по результатам экспериментов.

Таблица с данными экспериментов: [https://github.com/GidRadium/huffman-archiver/blob/main/research/benchmark\\_results.csv](https://github.com/GidRadium/huffman-archiver/blob/main/research/benchmark_results.csv).

Полученные графики: <https://github.com/GidRadium/huffman-archiver/blob/main/research/plots>.

## 6. Заключение

В ходе выполнения работы получены следующие результаты:

- выполнен обзор алгоритма Хаффмана и существующих подходов к построению архиваторов;
- разработано консольное приложение для сжатия и разжатия файлов по алгоритму Хаффмана;
- реализованы два режима архивирования, ориентированные на различные сценарии использования;
- разработан набор модульных и интеграционных тестов;
- проведено экспериментальное исследование эффективности и производительности реализации;
- подготовлена инфраструктура автоматической сборки, тестирования и генерации документации.

Репозиторий: <https://github.com/GidRadium/huffman-archiver>.

Техническая документация: <https://gidradium.github.io/huffman-archiver/>.

## 7. Литература

### Список литературы

- [1] Huffman D.A. A Method for the Construction of Minimum-Redundancy Codes // Proceedings of the IRE. 1952. Vol. 40, No. 9. P. 1098–1101.
- [2] Deutsch P. DEFLATE Compressed Data Format Specification version 1.3 (RFC 1951) [Электронный ресурс]. URL: <https://www.rfc-editor.org/rfc/rfc1951> (дата обращения: 16.06.2026).
- [3] Wikipedia contributors. Huffman coding [Электронный ресурс]. URL: [https://en.wikipedia.org/wiki/Huffman\\_coding](https://en.wikipedia.org/wiki/Huffman_coding) (дата обращения: 16.06.2026).